CRKL-Space

Database Product Document - CS 514 Phase 1 Draft

Team	Christopher (API Researcher), Kazuhiro (Database Developer), Lia (Team Lead / Frontend), Ryan (Backend)
Website	https://crkl-sdsu.github.io/CRKL/
Source code	https://github.com/CRKL-SDSU/CRKL
Date	October 19, 2026

1. Database Design

1.1 Entities

Our database centers around space exploration research and history, where that data is categorized based on various subjects. In this context, “subjects” include the following strong entities: **agencies, missions, launches,** and **spacecraft.** Here, the **missions** are the primary subject of our database, and are connected to the other entities via one or more of the following associative (junction) entities: **mission_agencies, mission_launches,** and **mission_spacecraft**.
As for what each entity, or table, contains, all entities contain some primary key in the form of their unique identifier (ID). However, the strong entities share the following attributes:

- A source_provider
- A source_id
- A name
- An image_url

In addition, each strong entity contains attributes unique within their own context. For example, **agencies** store any abbreviations (e.g., *NASA*) and their designated country_code; **launches** monitor updated statuses, as well as launch timeframes; and **spacecraft** store the specific spacecraft_type used.
Each junction table connects certain strong entities together and contains a composite primary key. This composite primary key consists of the IDs of the connected strong entities. For example, the composite primary key for **mission_launches** consists of the mission_id and launch_id (in other words, the primary keys from the respective tables). The only junction table to contain a new attribute is **mission_agencies**, which can store the role or purpose of that agency’s mission.

1.2 Entity Relationship Diagram (ERD)

The schema is mission-centric: four entity tables, with missions connected to the other three through junction tables.

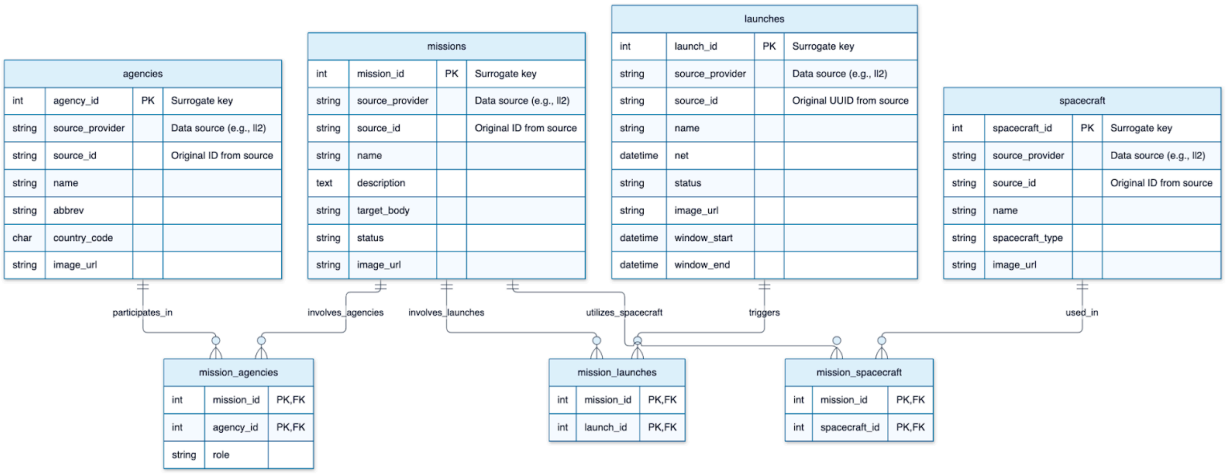


Figure 1. Entity-relationship diagram of the crkl_db schema.

Table	Purpose
missions	Mission name, description, target body, and status
agencies	Organization name, abbreviation, and country
spacecraft	Spacecraft name and type
launches	Launch name, no-earlier-than date (NET), launch window, and status
mission_agencies	Links missions and agencies, with each agency's role (e.g. Primary, Contributor)
mission_spacecraft	Links missions and the spacecraft they use
mission_launches	Links missions and the launches that carry them

Table 1. Tables in crkl_db.

We wanted our database to center around space missions because this sub-section of astronomy research is often overshadowed in comparison to everything else about space. The appeal of learning about *what* spectacular things we find in space often outpaces the interest in learning how far *we* can technically reach those spectacles. Aspiring researchers who want to get involved need to know both of these aspects, on top of other STEM-related fields, and our database aims to introduce and educate users on the niche side of information.

1.3 Keys and Constraints

- **Primary keys:** each strong entity has an `AUTO_INCREMENT` integer ID; each junction table uses a composite primary key, so the same link can't be stored twice.
- **Foreign keys:** both columns of each junction table reference their entity tables with `ON DELETE CASCADE`, so deleting a mission also removes its links.
- **Constraints:** `source_id` is `UNIQUE`, so the same source record can't be imported twice; required fields are `NOT NULL`; `role` defaults to *Primary*.

1.4 Normalization

Every relationship between missions and the other entities is many-to-many, and each one is resolved with a junction table, which keeps the schema in third normal form (3NF):

- **1NF:** every column holds one value. Each mission–agency pair is its own row, not a list.
- **2NF:** in `mission_agencies`, `role` depends on the whole composite key: NASA is Primary on one mission and a Contributor on another.
- **3NF:** no non-key column depends on another non-key column. An agency's name and country are stored only in `agencies`, never copied into `missions`.

2. Implementation

2.1 Architecture

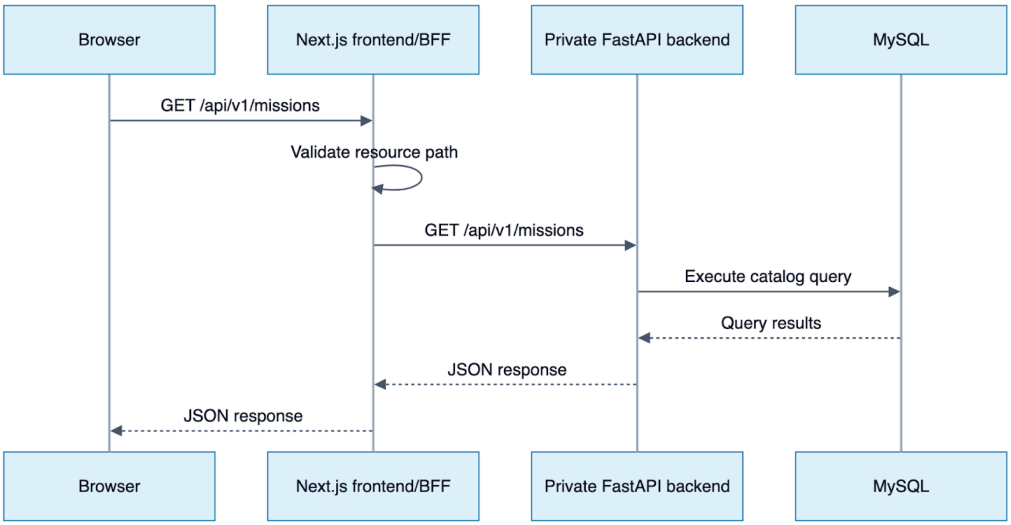


Figure 2. Sequence Diagram of the CRKL-Space Database.

We developed the frontend of our website with Next.js and React to serve as our Explore page, where users can input search requests that are forwarded to the backend as read-only requests. This way, the backend, which handles the API endpoints used for our data entries (e.g., searching, using related-record lookups), is never exposed to the browser and is protected from users. The backend itself connects to the “crkl_db” database as a dedicated “crkl” user, which can read and write rows but can't drop tables or change the schema.

2.2 REST API

Other parts of our application’s implementation rely on the REST (Representational State Transfer) API to query data entries in our database. These endpoints are as follows:

- `GET /api/v1/missions` | `/{id}`
- `GET /api/v1/agencies` | `/{id}`
- `GET /api/v1/spacecraft` | `/{id}`
- `GET /api/v1/launches` | `/{id}`

Each entry contains its corresponding ID; however, IDs aren’t necessary for certain query

commands. When acquiring a collection of entries, an ID is not needed; likewise, when acquiring a particular entry, an ID is needed.
On the topic of query commands, they themselves contain parameters that are used to affect the overall efficiency of our database:

- **Limit:** controls the pagination page size
- **Offset:** specifies the number of records to skip before retrieving data
- **Search:** enables pattern matching for particular entries (titles, agencies, etc.)
- **Related:** also matches entries through their linked records in the junction tables (e.g., `mission_agencies`)

2.3 Related-Record Search

When `related` is on, the backend adds `EXISTS` subqueries that follow the junction tables. A short excerpt of the query for a mission search, from `backend/code/endpoints.py`:

```
SELECT * FROM missions
WHERE name LIKE %s OR description LIKE %s
   OR target_body LIKE %s OR status LIKE %s
   OR EXISTS (SELECT 1 FROM mission_agencies ma
              JOIN agencies a ON a.agency_id = ma.agency_id
              WHERE ma.mission_id = missions.mission_id
                AND (a.name LIKE %s OR a.abbrev LIKE %s
                     OR a.country_code LIKE %s))
   -- similar EXISTS clauses for mission_spacecraft and mission_launches
ORDER BY mission_id
LIMIT %s OFFSET %s;
```

Example: searching missions for “NASA” returns 0 rows from the missions' own columns, but 28 with related search: the missions linked to NASA in `mission_agencies`.

Every user value is sent as a query parameter (`%s`), never concatenated into SQL, and the `LIKE` wildcards in the search term are escaped, which protects against SQL injection.

2.4 Testing

- **Backend** (pytest, with a mocked database): checks the generated SQL and the API responses.
- **Frontend** (Vitest): checks search, the detail panel, and Back/Forward navigation.
- **GitHub Actions** runs both suites, together with a production build of the frontend, on every push and pull request.

3. Course Concepts Applied

Concept	Where it appears in CRKL-Space
SQL queries	`SELECT` with `WHERE`, `LIKE`, `ORDER BY`, `LIMIT`/`OFFSET`; joins and `EXISTS` subqueries; `COUNT(*)` for pagination
Entity-Relationship Diagram (ERD) modeling	The ERD (Figure 1)
Relational database design	Four strong entities connected through three junction tables
Primary, composite, and foreign keys	IDs on strong entities; composite keys and foreign keys on junction tables
Normalization and Third Normal Form (3NF)	Many-to-many relationships resolved into junction tables; `role` stored on the relationship
Attribute constraints	`NOT NULL`, `UNIQUE` on `source_id`, `DEFAULT` for `role`, `ON DELETE CASCADE`
Database programming	Python (FastAPI) backend running parameterized SQL

Table 2. Course concepts and where they appear.

4. Screenshots

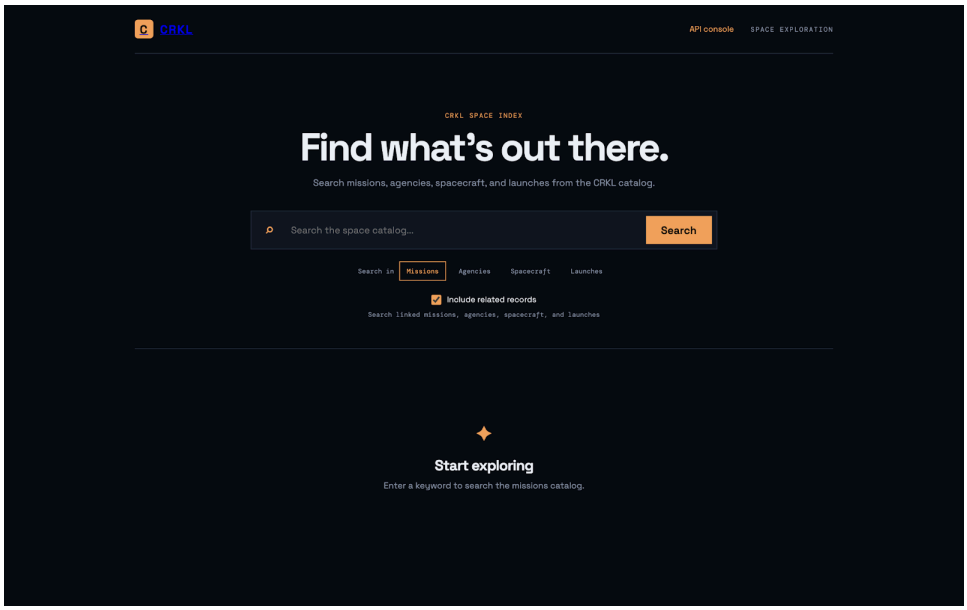


Figure 3. Explore page before a search.

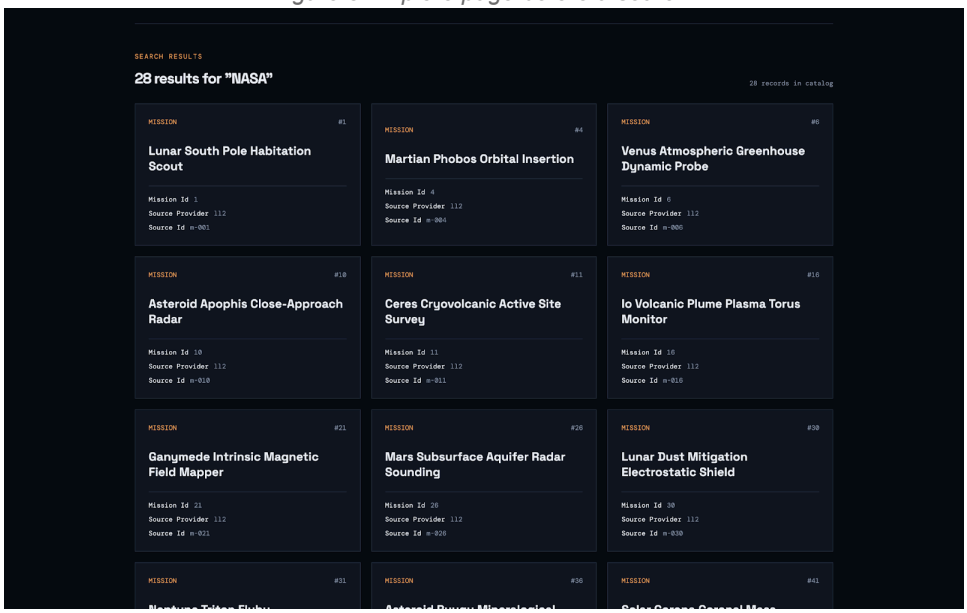


Figure 4. Search results for missions with the keyword "NASA" and related records included.

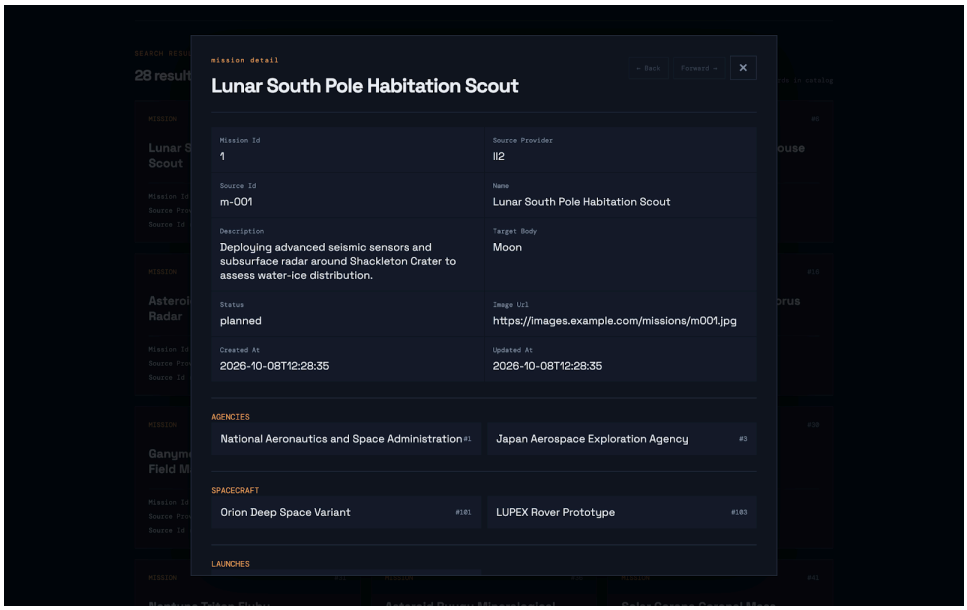


Figure 5. Detail panel for a mission, showing its agencies, spacecraft, and launches.

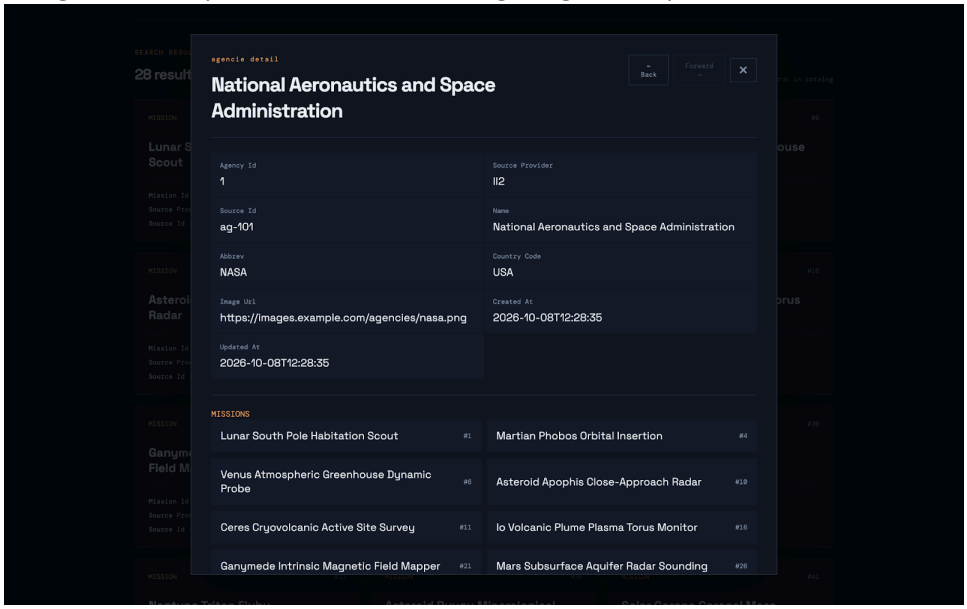


Figure 6. Detail panel for an agency, opened from the mission panel. The Back button is enabled.

5. User Guide

5.1 Start the Application

1. Install and start Docker Desktop.
2. Get the code: `git clone --branch phase1 https://github.com/CRKL-SDSU/CRKL.git`, then open the `CRKL` folder.
3. Copy `.env.example` to `.env` and replace its two placeholder passwords with new ones of your own choice (any values work).
4. Run `docker compose up --build`, then open `http://localhost:3000/explore`.

Full details, including the configuration files and troubleshooting: Source Code & Executable.

5.2 Search the Catalog

1. Under the search box, choose what to search: **Missions**, **Agencies**, **Spacecraft**, or **Launches**.
2. Type a keyword, such as *NASA* or *Moon*, and press **Search**. Search is case-insensitive and matches anywhere in a field. Leave the box empty to list everything.
3. Keep **Include related records** checked to also find records through their links (for example, every mission NASA takes part in). Uncheck it to match only the selected type's own fields.

5.3 View Details and Follow Links

1. Click a result card to open its detail panel, with all its fields and related records.
2. Click a related record to open it in the same panel.
3. Use **Back** and **Forward** to retrace your steps. Close the panel with ×, by clicking outside it, or with Esc.

The complete guide, including the fields searched for each type and the messages you may see, is on the Instructions page.